## Tabla de contenido


***Desafío de Desarrollo en Python*** ..... ***3***

**Sistema de Gestión de Pedidos de Clientes** ..... **3**

***Historia de Usuario*** ..... ***3***

***Contexto del Problema*** ..... ***3***

***Restricciones Técnicas*** ..... ***4***

***Estructuras de Datos*** ..... ***4***

***Uso de Funciones*** ..... ***4***

***Organización del Código*** ..... ***5***

***Funcionalidades de Negocio*** ..... ***5***

**1. Registro de Clientes** ..... **5**

**2. Registro de Productos** ..... **6**

**3. Creación de Pedidos** ..... **6**

**4. Consulta de Pedidos Registrados** ..... **6**

**5. Cálculo de Ingresos del Día** ..... **6**

**6. Generación de Reporte Final** ..... **7**

***Proceso de Desarrollo*** ..... ***7***

***Gestión del Proyecto en Azure DevOps*** ..... ***7***

1. Product Backlog ..... 7
2. Registro de Tareas ..... 7

***Criterios de Aceptación*** ..... ***8***

**1. Diseño de la Solución** ..... **8**

**2. Programa Funcional** ..... **9**

**3. Uso Correcto de Estructuras de Datos** ..... **9**

**4. Modularización** ..... **9**

5. Documentación ..... 9

*Entregables del Proyecto* ..... *10*

1. Repositorio en GitHub ..... 10

2. Archivo README (en inglés) ..... 10

3. Código Documentado ..... 10

4. Diagrama de Flujo ..... 11

5. Evidencia de Gestión en Azure DevOps ..... 11

*Nota Importante* ..... *11*

*Reglas para la Presentación y Sustentación del Programa en Python* ..... *12*

1. Conformación de los equipos ..... 12

*2. Sistema de Evaluación* ..... *12*

*Penalización por ayudas* ..... *12*

Ejemplo ..... 13

*3. Penalización por preguntas del evaluador* ..... *13*

Ejemplo ..... 13

*4. Participación del equipo* ..... *14*

*5. Criterios que serán evaluados* ..... *14*

*7. Objetivo del reto* ..... *14*

# Desafío de Desarrollo en Python

## Sistema de Gestión de Pedidos de Clientes

# Historia de Usuario

**Título:** Automatización del registro y procesamiento de pedidos de clientes

**Como** gerente de operaciones comerciales **Quiero** un sistema desarrollado en Python que permita registrar y procesar pedidos de clientes **Para** poder llevar un control estructurado de los productos vendidos, calcular automáticamente los totales de cada pedido y obtener reportes consolidados de las ventas realizadas durante la jornada.

# Contexto del Problema

Una empresa distribuidora actualmente registra los pedidos de sus clientes de forma manual utilizando hojas de cálculo y registros en papel. Este proceso genera varias dificultades operativas:

- No existe un control claro de qué clientes han realizado pedidos.
- Es difícil identificar qué productos fueron vendidos durante el día.
- Los cálculos de totales por pedido se realizan manualmente.
- No se puede generar fácilmente un reporte consolidado de las ventas diarias.

Por esta razón, se requiere desarrollar un **script en Python** que permita registrar pedidos de clientes durante la jornada laboral y generar un reporte estructurado al finalizar la ejecución del programa.

Cada pedido debe contener como mínimo:

- Identificador del pedido
- Nombre del cliente
- Producto solicitado
- Precio unitario
- Cantidad comprada

El sistema debe permitir registrar múltiples pedidos durante la ejecución del programa y generar un resumen consolidado al finalizar.

# Restricciones Técnicas

El desarrollo debe cumplir obligatoriamente con las siguientes reglas.

# Estructuras de Datos

El sistema debe utilizar exclusivamente:

- **Diccionarios**
- **Tuplas**

No se permite utilizar:

- **Listas**

Ejemplo de estructuras permitidas:

**Tupla**

Para representar información inmutable de productos.

```
(product_id, product_name, unit_price)
```

**Diccionario**

Para almacenar pedidos o clientes registrados.

# Uso de Funciones

El sistema debe estar construido utilizando **funciones reutilizables**.

Reglas obligatorias:

- Todas las funciones deben **recibir parámetros**.
- Todas las funciones deben **retornar datos**.
- No se permiten funciones que únicamente impriman resultados.
- Las funciones deben ser **reutilizables** y no estar diseñadas para un único caso específico.

# Organización del Código

El programa debe estar **modularizado**.

Cada módulo debe encargarse de responsabilidades específicas del sistema.

# Funcionalidades de Negocio

- El sistema debe incluir **seis funcionalidades principales**.
- Cada funcionalidad debe ser desarrollada por **un integrante del equipo**.
- Cada grupo estará conformado por **6 integrantes**.

## 1. Registro de Clientes

Permitir registrar clientes dentro del sistema.

La información debe almacenarse en un **diccionario**.

Datos requeridos:

- ID del cliente
- Nombre del cliente
- Correo electrónico

## 2. Registro de Productos

Permitir registrar productos disponibles para la venta.

La información de los productos debe almacenarse en **tuplas**.

Ejemplo:

```
(product_id, product_name, unit_price)
```

## 3. Creación de Pedidos

Permitir crear un pedido asociando:

- Cliente
- Producto
- Cantidad

El sistema debe calcular automáticamente el valor total del pedido.

```
total_pedido = precio_unitario * cantidad
```

## 4. Consulta de Pedidos Registrados

Permitir visualizar los pedidos registrados en el sistema mostrando:

- Cliente
- Producto
- Cantidad
- Total del pedido

## 5. Cálculo de Ingresos del Día

El sistema debe calcular el **total de ingresos generados durante la jornada**, sumando todos los pedidos registrados.

### 6. Generación de Reporte Final

El sistema debe generar un reporte que incluya:

- Total de pedidos registrados
- Total de ingresos generados
- Pedidos agrupados por cliente
- Productos vendidos durante el día

# Proceso de Desarrollo

El desarrollo del proyecto debe seguir un enfoque de **trabajo colaborativo utilizando prácticas ágiles**.

Cada integrante será responsable de **una funcionalidad del sistema**.

# Gestión del Proyecto en Azure DevOps

Cada equipo deberá gestionar el desarrollo utilizando **Azure DevOps**.

Se debe crear:

### 1. Product Backlog

Debe incluir la **historia de usuario principal**.

### 2. Registro de Tareas

Cada funcionalidad debe registrarse como una **tarea en Azure DevOps**.

Ejemplo de tareas:

- Implementar registro de clientes
- Implementar registro de productos
- Implementar creación de pedidos
- Implementar consulta de pedidos
- Implementar cálculo de ingresos
- Implementar módulo de reportes

Cada tarea debe incluir:

- Descripción de la tarea
- Desarrollador asignado
- Estado de avance

# Criterios de Aceptación

El desarrollo será considerado completo si cumple con los siguientes criterios.

## 1. Diseño de la Solución

Antes de iniciar el desarrollo del código, el equipo debe diseñar un **diagrama de flujo** que represente la lógica del sistema.

El diagrama debe incluir:

- Inicio del programa
- Interacción con el usuario
- Registro de datos
- Procesamiento de pedidos
- Generación del reporte final
- Finalización del programa

El desarrollo del código solo podrá comenzar después de validar el diagrama.

## 2. Programa Funcional

El programa debe ejecutarse desde la **terminal** y permitir al usuario:

- Registrar clientes
- Registrar productos
- Crear pedidos
- Consultar pedidos
- Generar reportes

## 3. Uso Correcto de Estructuras de Datos

El programa debe demostrar el uso correcto de:

- **Diccionarios**
- **Tuplas**

El uso de **listas está estrictamente prohibido**.

## 4. Modularización

El código debe estar organizado en módulos y funciones reutilizables.

## 5. Documentación

El código debe incluir:

- Comentarios explicativos
- Descripción de funciones
- Parámetros de entrada
- Valores de retorno

Toda la documentación del código debe estar **en inglés**.

# Entregables del Proyecto

Cada equipo deberá entregar:

## 1. Repositorio en GitHub

El repositorio debe contener:

- Código fuente del proyecto
- Estructura modular del sistema

## 2. Archivo README (en inglés)

El README debe incluir:

- Descripción del proyecto
- Arquitectura del sistema
- Instrucciones para ejecutar el programa
- Explicación de las estructuras de datos utilizadas
- Descripción de los módulos implementados

## 3. Código Documentado

El código debe incluir:

- Comentarios claros
- Documentación de funciones
- Variables con nombres en inglés

## 4. Diagrama de Flujo

Documento que represente la lógica del sistema.

## 5. Evidencia de Gestión en Azure DevOps

Se debe presentar evidencia de:

- Historia de usuario creada
- Tareas registradas
- Asignación de tareas por integrante
- Seguimiento del progreso

# Nota Importante

Este problema puede resolverse de múltiples maneras. Antes de iniciar su desarrollo, el equipo debe definir claramente:

- Cómo se almacenará la información
- Cómo se relacionarán las estructuras de datos
- Cómo se dividirán las responsabilidades entre módulos

# Reglas para la Presentación y Sustentación del Programa en Python

## 1. Conformación de los equipos

1. Cada equipo estará conformado por **6 integrantes**.
2. Cada equipo debe designar obligatoriamente:
    - **1 Líder del equipo**
    - **1 Sustentador**
3. El **líder** será responsable de:
    - Coordinar el trabajo del grupo.
    - Garantizar que todos los integrantes participen en el desarrollo.
4. El **sustentador** será el encargado de:
    - Presentar el programa.
    - Explicar la **lógica del código**.
    - Explicar **cómo fue desarrollado el programa**.
    - Responder las preguntas del evaluador.

# 2. Sistema de Evaluación

Cada equipo inicia la sustentación con **100 puntos**.

El objetivo del equipo es **mantener los 100 puntos durante toda la sustentación**.

Los puntos pueden disminuir según las siguientes reglas.

## Penalización por ayudas

Durante la sustentación, el sustentador **puede solicitar ayuda a su equipo**.

Sin embargo, cada ayuda tendrá una penalización:

- **Primera ayuda:** -10 puntos
- **Segunda ayuda:** -10 puntos adicionales
- **Tercera ayuda:** -10 puntos adicionales
- Y así sucesivamente.

### Ejemplo

Puntaje inicial: **100 puntos**

| Ayuda solicitada | Puntaje restante |
|---|---|
| 1 ayuda | 90 puntos |
| 2 ayudas | 80 puntos |
| 3 ayudas | 70 puntos |

# 3. Penalización por preguntas del evaluador

Durante la sustentación, el evaluador realizará preguntas relacionadas con:

- La lógica del programa
- La estructura del código
- Funciones utilizadas
- Algoritmos implementados
- Decisiones de diseño

Si el sustentador **no responde correctamente una pregunta**, el equipo perderá:

**-20 puntos por cada pregunta incorrecta**

### Ejemplo

Puntaje inicial: **100 puntos**

| Situación | Puntaje |
|---|---|
| 1 pregunta incorrecta | 80 puntos |
| 2 preguntas incorrectas | 60 puntos |
| 3 preguntas incorrectas | 40 puntos |

# 4. Participación del equipo

Aunque solo **una persona sustenta**, todos los integrantes del equipo deben:

- Conocer el funcionamiento del programa
- Haber participado en su desarrollo
- Estar preparados para apoyar al sustentador si es necesario

# 5. Criterios que serán evaluados

Durante la sustentación se evaluará:

1. **Funcionamiento del programa**
2. **Claridad en la explicación del código**
3. **Comprensión de la lógica implementada**
4. **Uso adecuado de Python**
5. **Organización del programa**
6. **Capacidad para responder preguntas técnicas**

# 7. Objetivo del reto

El reto principal es:

**Mantener los 100 puntos durante toda la sustentación.**

Para lograrlo, el sustentador deberá:

- Explicar claramente el programa
- Demostrar dominio del código
- Responder correctamente las preguntas
- Minimizar la necesidad de ayuda del equipo